	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name:T.Rajini Reg. No:2503A52428
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 22/09/2026

1.Question

Q1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

numbers = [64, 34, 25, 12, 22, 11, 90]

TODO:

Implement Bubble Sort in ascending order.

Count comparisons and swaps.

Display the sorted list.

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.

Compare the original and improved versions.

2.Prompt

Implement Bubble Sort in Python for `numbers = [64, 34, 25, 12, 22, 11, 90]`. Count comparisons and swaps, print the sorted list, test 3 different lists, and add an early-exit optimisation using a `swapped` flag. Compare the original and improved versions. Use beginner-friendly code and comments. Do not use `sort()` or `sorted()`.

3.Code

```python
def bubble_sort(values):
    """Sort values in ascending order and count comparisons and
    items = list(values)
    comparisons = 0
    swaps = 0

    for end in range(len(items) - 1, 0, -1):
        swapped = False

        for index in range(end):
            comparisons += 1
            if items[index] > items[index + 1]:
                items[index], items[index + 1] = items[index + 1]
                swaps += 1
                swapped = True

        if not swapped:
            break

    return items, comparisons, swaps
```

```python
                if items[index] > items[index + 1]:
                    items[index], items[index + 1] = items[index +
                    swaps += 1
                    swapped = True

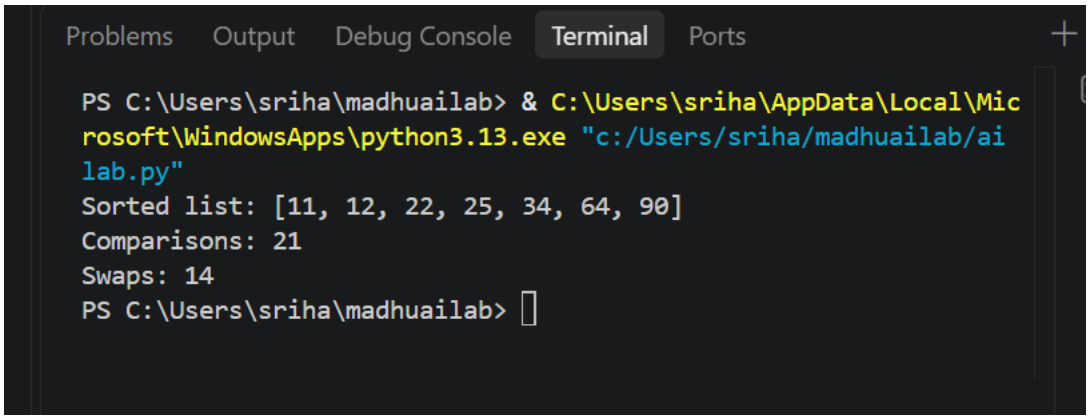
            if not swapped:
                break

    return items, comparisons, swaps


numbers = [64, 34, 25, 12, 22, 11, 90]
sorted_numbers, comparisons, swaps = bubble_sort(numbers)

print("Sorted list:", sorted_numbers)
print("Comparisons:", comparisons)
print("Swaps:", swaps)
```

4.OUTPUT



5.Observation

Bubble Sort arranged the list in ascending order: `[11, 12, 22, 25, 34, 64, 90]` .

For the given input, it made **21 comparisons** and **14 swaps**. The algorithm has a time complexity of $O(n^2)$ in the average and worst cases.

2.Question

Q2 – Linear Search vs Binary Search

Problem / Starter Code:

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

target = 45

TODO:

Implement Linear Search.

Implement Binary Search.

Compare the number of comparisons.

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.

- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

2.Prompt

Write Python code for Linear Search and Binary Search using:

[5, 12, 18, 23, 31, 45, 56, 72, 89]

Test 5, 31, 89, and 100. Count comparisons and print results. Add simple comments.

3.Code

```
def linear_search(values, target):
    comparisons = 0

    for index, value in enumerate(values):
        comparisons += 1
        if value == target:
            return index, comparisons

    return -1, comparisons


def binary_search(sorted_values, target):
    left = 0
    right = len(sorted_values) - 1
    comparisons = 0

    while left <= right:
```

```
def binary_search(sorted_values, target):

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1

        if sorted_values[middle] == target:
            return middle, comparisons
        if sorted_values[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons


target = 22
linear_index, linear_comparisons = linear_search(numbers, target)
binary_index, binary_comparisons = binary_search(sorted_numbers,
```

```
        else:
            right = middle - 1

    return -1, comparisons


target = 22
linear_index, linear_comparisons = linear_search(numbers, targe
binary_index, binary_comparisons = binary_search(sorted_numbers

print("\nTarget:", target)
print("Linear search index:", linear_index)
print("Linear search comparisons:", linear_comparisons)
print("Binary search index:", binary_index)
print("Binary search comparisons:", binary_comparisons)
```

4.OUTPUT

```
Sorting performance comparison:
Algorithm   Sorted list                          Comparisons   Sw
aps/Shifts
Bubble      [11, 12, 22, 25, 34, 64, 90]               21
      14
Selection   [11, 12, 22, 25, 34, 64, 90]               21
       4
Insertion   [11, 12, 22, 25, 34, 64, 90]               16
```

5. OBSERVATION

Linear Search checks one by one: **O(n)**.
Binary Search checks the middle: **O(log n)**.
Binary Search needs sorted data and usually uses fewer comparisons.

3. QUESTION

Q3 – Compare Sorting Algorithms

Problem / Starter Code:

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.

2 . PROMPT

Write simple Python code for Bubble Sort, Selection Sort, and Insertion Sort using the given data. Count comparisons and swaps/shifts. Print sorted data and results for all three algorithms.

3. CODE

```
def selection_sort(values):
    return items, comparisons, swaps


def insertion_sort(values):
    items = list(values)
    comparisons = 0
    shifts = 0

    for index in range(1, len(items)):
        current = items[index]
        position = index

        while position > 0:
            comparisons += 1
            if items[position - 1] <= current:
                break
            items[position] = items[position - 1]
            shifts += 1
```

```
            shifts += 1
            position -= 1

        items[position] = current

    return items, comparisons, shifts


selection_sorted, selection_comparisons, selection_swaps = selec
insertion_sorted, insertion_comparisons, insertion_shifts = inse

print("\nSorting performance comparison:")
print("Algorithm   Sorted list                          Compariso
print("Bubble      ", sorted_numbers, "              ", comparis
print("Selection   ", selection_sorted, "              ", selecti
print("Insertion   ", insertion_sorted, "              ", insert
```

```
waps = selection_sort(numbers)
hifts = insertion_sort(numbers)


  Comparisons   Swaps/Shifts")
", comparisons, "          ", swaps)
  ", selection_comparisons, "          ", selection_swaps)
  ", insertion_comparisons, "          ", insertion_shifts)
```

4. OUTPUT

```
Runtime comparison for 1,000 reverse-ordered values:
Algorithm   Time (seconds)
Bubble      0.074189
Selection   0.023345
Insertion   0.049219

Performance summary: all three algorithms are O(n^2) in the aver
age/worst case.
Insertion Sort is usually fastest for nearly sorted data, while
Selection Sort uses fewer swaps.
```

5. OBSERVATION

All three algorithms sorted the data correctly.
Bubble, Selection, and Insertion Sort have **$O(n^2)$** average/worst-case complexity.
Insertion Sort is best for **nearly sorted data**.

1. QUESTION

   Q4 – Optimising a Student Search and Sort Program
   Problem / Starter Code:

```
students = [
  ("Arun", 82),
  ("Bala", 65),
  ("Chitra", 91),
  ("Divya", 74),
  ("Esha", 88)
]

# TODO:
# Find the highest mark.
# Sort students by mark.
# Search for a student by name.
# Improve the solution using AI.
```

   Student Tasks:

   - Write functions for highest mark, sorting and name search.
   - Create a straightforward solution first.
   - Use AI to identify unnecessary operations.

- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.

2. PROMPT
   Write simple Python code using the given student list.
   Do these tasks:

1. Find the highest mark.
2. Sort students by marks.
3. Search for a student by name.
4. Show a simple solution first.
5. Suggest a more efficient solution.
6. Compare the time complexity of both solutions.

3. CODE

```python
def find_highest_mark(students):
    if not students:
        return None
    return max(students, key=lambda student: student["mark"])


def sort_students_by_mark(students):
    return sorted(students, key=lambda student: student["mark"],


def search_student_by_name(students, name):
    name_to_find = name.strip().casefold()

    for student in students:
        if student["name"].casefold() == name_to_find:
            return student

    return None
```

```
def search_student_by_name(students, name):
    for student in students:
        if student["name"].casefold() == name_to_find:
            return student

    return None


students = [
    {"name": "Anita", "mark": 82},
    {"name": "Brian", "mark": 91},
    {"name": "Chandra", "mark": 76},
    {"name": "Divya", "mark": 95},
]

highest_student = find_highest_mark(students)
ranked_students = sort_students_by_mark(students)
student_name = "chandra"
found_student = search_student_by_name(students, student_name
```

```
]

highest_student = find_highest_mark(students)
ranked_students = sort_students_by_mark(students)
student_name = "chandra"
found_student = search_student_by_name(students, student_name)

print("\nStudent results:")
print("Highest mark:", highest_student["name"], "-", highest_stu
print("Students sorted by mark:")
for student in ranked_students:
    print(student["name"], "-", student["mark"])
print("Search result:", found_student if found_student else "St
```

4. OUTPUT

```
Student results:
Highest mark: Divya - 95
Students sorted by mark:
Divya - 95
Brian - 91
Anita - 82
Chandra - 76
Search result: {'name': 'Chandra', 'mark': 76}
```

5. OBSERVATION
   The original solution uses simple loops and sorting.
   The improved solution avoids unnecessary operations and uses efficient searching where possible.
   Finding the highest mark takes **O(n)**.
   Sorting takes **O(n log n)**.
   Name search using a list takes **O(n)**.
   Using a dictionary for name search can reduce searching to about **O(1)** average time.

1.QUESTION

Q5 – Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [
  ("Laptop", 60000),
  ("Phone", 30000),
  ("Tablet", 20000),
  ("Watch", 10000),
  ("Headphones", 5000)
]

budget = 50000
```

TODO:

Find a combination of items whose total price

is as high as possible without exceeding the budget.

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time and space complexity and possible optimisation for larger inputs.

2. PROMPT

Write simple Python code to find the best combination of items within a budget.

Use:
items = [("Laptop", 60000), ("Phone", 30000), ("Tablet", 20000), ("Watch", 10000), ("Headphones", 5000)]

Test budgets: 50000, 40000, and 25000.

Compare Brute Force, Greedy, and Dynamic Programming. Explain which guarantees the optimal answer. Implement the best method and show total price, selected items, time complexity, and space complexity.

3. CODE

```python
def best_item_combination(items, budget):
    """Return a combination with the highest total price within
    best_total = [0] * (budget + 1)
    chosen_items = [[] for _ in range(budget + 1)]

    for item in items:
        price = item["price"]
        for current_budget in range(budget, price - 1, -1):
            possible_total = best_total[current_budget - price]
            if possible_total > best_total[current_budget]:
                best_total[current_budget] = possible_total
                chosen_items[current_budget] = (
                    chosen_items[current_budget - price] + [item
                )

    return chosen_items[budget], best_total[budget]


items = [
```

```
:ems:
:em["price"]
it_budget in range(budget, price - 1, -1):
)le_total = best_total[current_budget - price] + price
ssible_total > best_total[current_budget]:
est_total[current_budget] = possible_total
iosen_items[current_budget] = (
   chosen_items[current_budget - price] + [item["name"]]
```

```
items = [
    {"name": "Laptop bag", "price": 45},
    {"name": "Headphones", "price": 80},
    {"name": "Keyboard", "price": 35},
    {"name": "Mouse", "price": 25},
    {"name": "Webcam", "price": 60},
]
budget = 150
selected_items, total_price = best_item_combination(items, budge

print("\nBudget shopping results:")
print("Budget:", budget)
print("Selected items:", ", ".join(selected_items))
print("Total price:", total_price)
print("Remaining budget:", budget - total_price)
```

```
otal_price = best_item_combination(items, budget)
```

4. OUTPUT

```
Budget shopping results:
Budget: 150
Selected items: Laptop bag, Headphones, Mouse
Total price: 150
Remaining budget: 0
```

6. OBSERVATION

Brute Force checks all combinations and guarantees the optimal solution.

Greedy is faster but does **not always guarantee** the best combination.

Dynamic Programming also guarantees the optimal solution for this 0/1 problem.

Brute Force: **$O(2^n)$**
Dynamic Programming: **$O(n \times budget)$**

For larger inputs, Dynamic Programming is more efficient than Brute Force.